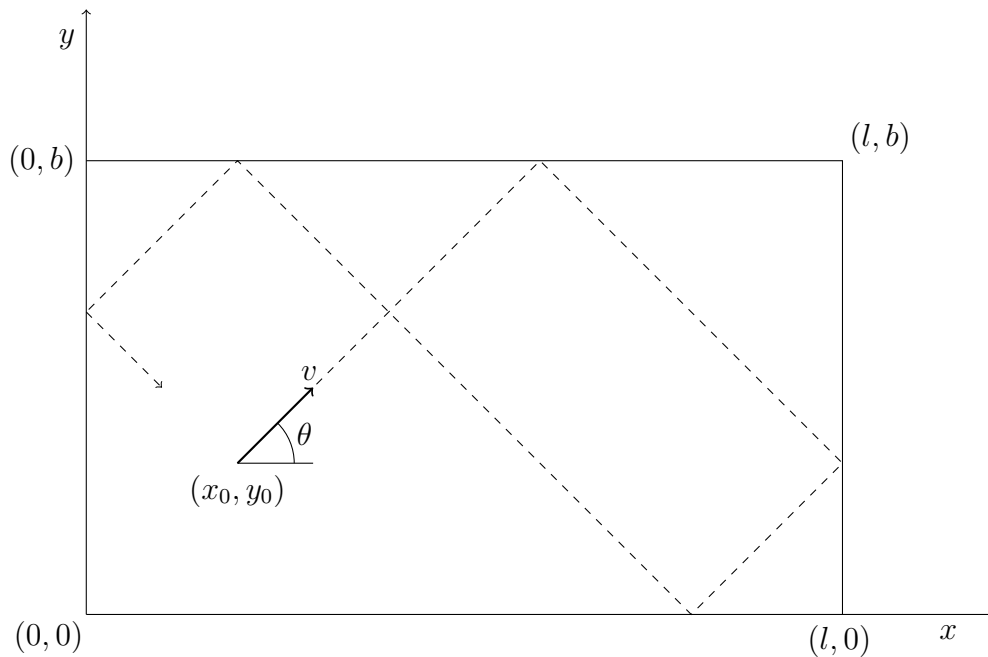


Models and Tools for Cyber-Physical Systems
Exercise sheet 11
WITH SOLUTIONS

Exercise 1: Billiards table



Consider a billiards table as schematically depicted above. We want to create a hybrid model to predict the path of the ball after being hit by the player. Assume that the table has a length l and breadth b and that the ball is laying at rest at (x_0, y_0) . A player hits the ball under angle θ giving the ball an initial speed v in that direction.

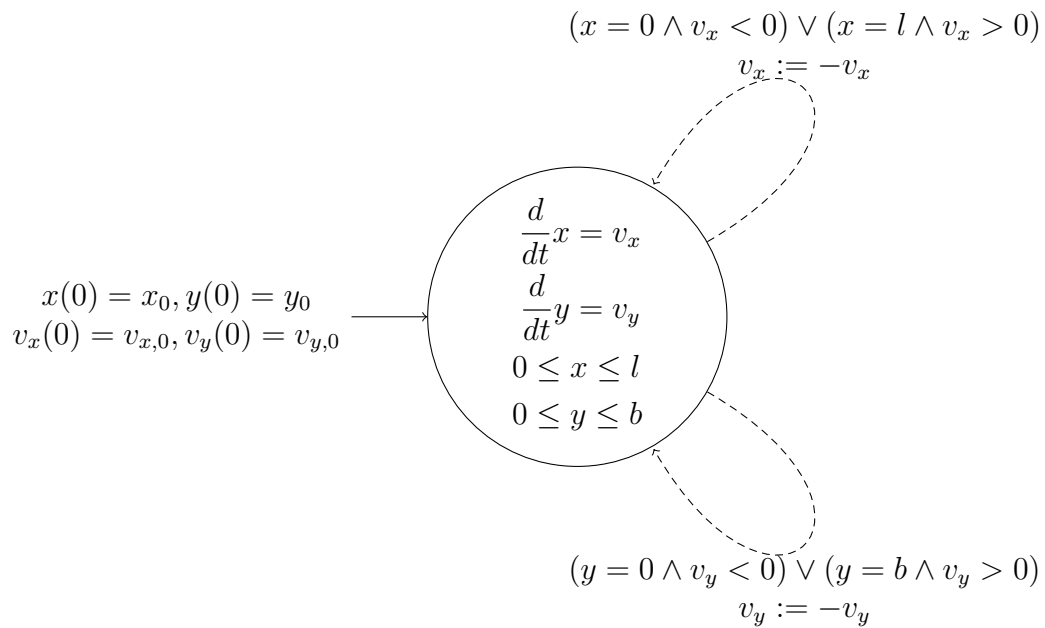
For simplicity, we assume that the ball has perfect collisions with the billiard table's sides (i.e., no energy is lost). That means that if the ball hits one of the sides parallel to the x -axis, its velocity in the y -direction flips sign while its velocity in the x -direction remains unchanged; similarly, if the ball hits one of the sides parallel to the y -direction, its velocity in the x -direction flips sign while its velocity in the y -direction remains unchanged.

- What is the initial velocity of the ball in the x -direction, $v_{x,0}$, and what is the initial velocity of the ball in the y -direction, $v_{y,0}$? Express those velocities in terms of v and θ .
- Construct a hybrid automaton capturing the motion of the ball on this perfect billiards table.

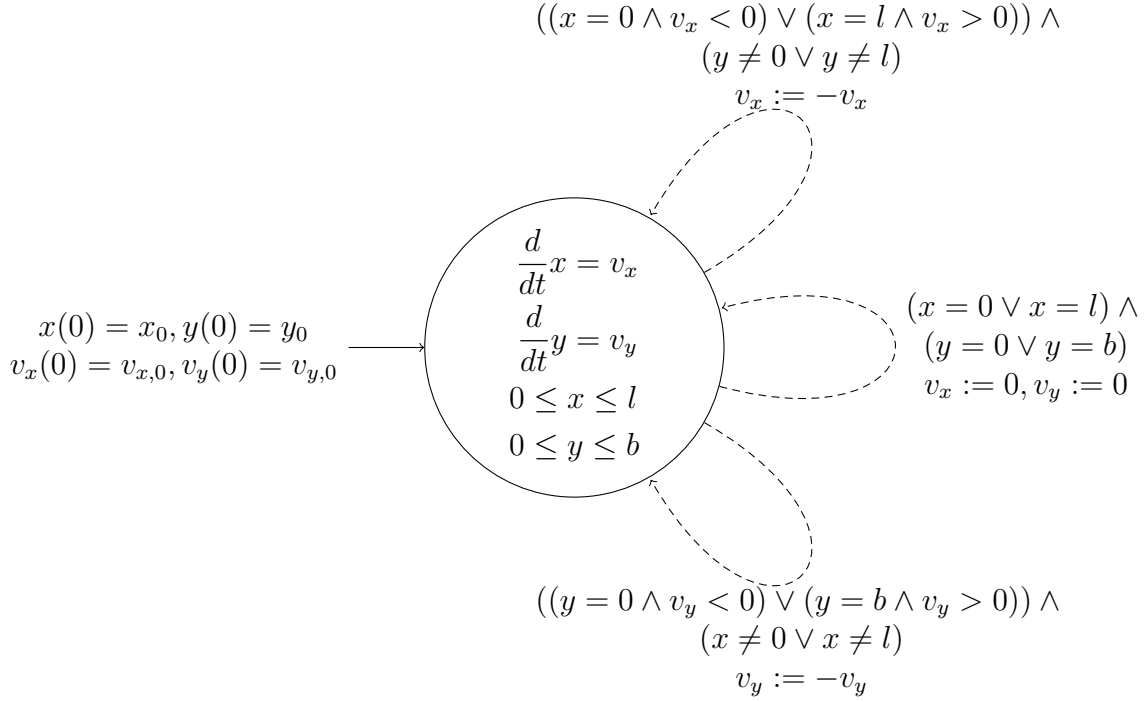
- (c) When the ball reaches one the four corners, it drops in a hole and stops moving. Adjust your hybrid automaton to include this situation.

.....Solution sketch

- (a) The initial velocity of the ball in the x -direction can be expressed as $v_{x,0} = v \cos(\theta)$. The initial velocity of the ball in the y -direction can be expressed as $v_{y,0} = v \sin(\theta)$.
- (b) One solution could be the hybrid automaton shown below. Instead of using a synchronization channel, conditions on the velocity are used to prevent Zeno behavior when a ball hits a side.



- (c) Two adjustments are needed: a new transition needs to be added to catch the corners, and the transitions from (b) need to become invalid in case of the ball reaches the corner to prevent the ball from leaving the corner instead of dropping.



Exercise 2: Cruise Control

Consider the car model from the course

$$\frac{d}{dt}x(t) = v(t) \qquad \frac{d}{dt}v(t) = -\frac{k}{m}v(t) + \frac{1}{m}F(t),$$

where x is the position, v the velocity and F the force applied in case of mass m and friction coefficient k . Starting at position $x(0) = 0$, we wish to accelerate the car to the speed of 20 (m/s) using a proportional controller with appropriate gain K_p for $m = 1000$ (kg) and $k = 50$.

- Assuming that we have no feedforward control and measure only the velocity of the car, provide a continuous-time component in block diagram form.
- Create a continuous-time component of the proportional controller in block diagram form and indicate how the controller component is connected to the car model component. Finally, combine them together into a single continuous-time component.
- Using the numerical solver from the previous sheet, tune your controller, i.e., find an appropriate value for K_p , such that the car attains the speed of $v = 20$ (m/s) within $t = 100$ (s). Describe the quality of your controller in terms of steady state error, settling time, overshoot and rise time.
- Based on your observations from the pure proportional controller, design a PI controller that improves the results. Tune your chosen PI architecture.

Assume now that there is a car ahead whose position at t (s) is given by $y(t) = 150 + 15t$ (m). We wish to extend the foregoing continuous-time component to a hybrid component which

- stops accelerating as soon as the distance to the car ahead falls below 50 (m).
- activates the previous controller once the car ahead is more than 100 (m) away.

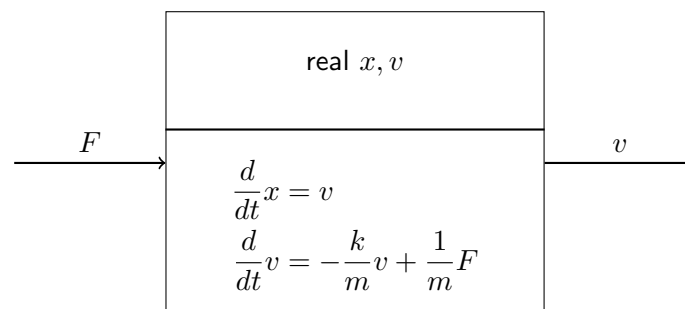
(e) Create the aforementioned hybrid component in block diagram form.

(f) Implement the hybrid controller by extending the numerical solver from previous sheets. By running simulations, decide whether the performance of your controller is good. If not, argue why not and try to improve it. [Hint: May it be beneficial to reset the integral error for certain mode changes?]

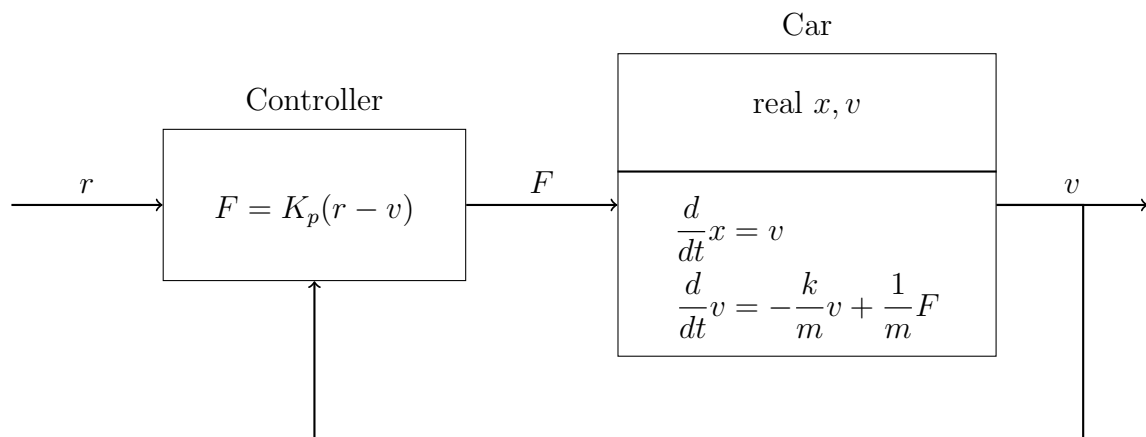
(g) Assume now that the controller does not stop accelerating the car once the distance to the car ahead falls below 50 (m). What is the maximal force F_h with which we could accelerate the car before a crash occurs?

.....Solution sketch

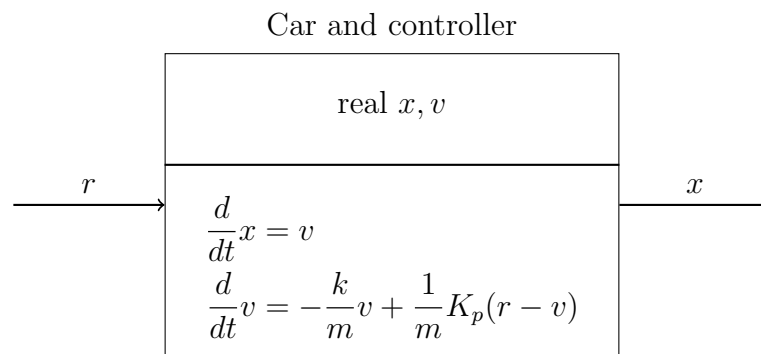
(a) The model is already given as a set of first-order differential equations, so we do not have to do any rewriting. The continuous-time component of the car is shown below.



(b) The continuous-time component of the controller is shown below. Note that a proportional controller does not have an internal state.



The combined continuous-time component of the car and the controller is shown below.



- (c) You can adapt the solution code from the previous sheet where you change function `f` as follows.

```

1 def f(x, t):
2     m = 1000
3     k = 50
4     Kp = 300
5     r = 20
6     der_x = x[1]
7     der_v = -k / m * x[1] + Kp * (r - x[1]) / m
8     return [der_x, der_v]
9

```

Figure 1 shows a plot of the velocity v (blue) for a gain of $K_p = 300$.

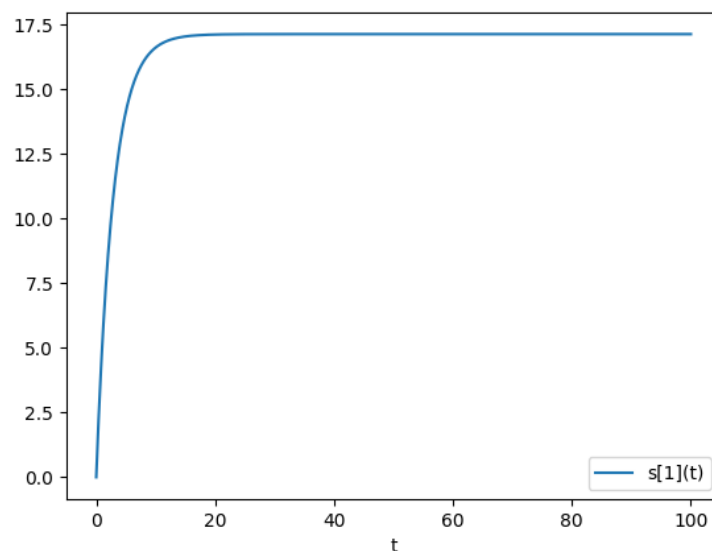


Figure 1: Car model state trajectory with only proportional controller $K_p = 300$.

One can observe that the proportional controller results in no overshoot but a significant steady-state error. The magnitude of the gain K_p determines the settling time (higher values of K_p result in shorter settling times) and the steady-state error (higher values of K_p result

in smaller steady-state errors) of the controller system. In theory, one needs to have $K_p = \infty$ to have no steady-state error, but that is unrealistic for actual implementation.

- (d) The steady-state error observed in the previous question can be solved by introducing the integral controller. To implement the integral controller, we need to integrate the error. We can use the following observation to implement this integral in numerical solvers.

$$\begin{aligned} \int e \, dt &= I && \text{need to calculate } I \\ \frac{d}{dt} \int e \, dt &= \frac{d}{dt} I && \text{differentiate both sides} \\ e &= \frac{d}{dt} I && \text{simplifying the equation} \end{aligned}$$

Observe that we effectively have created an additional internal state variable (for the controller), see also the block diagram in Figure 6.15 in the book of Alur.

You can adapt the solution code from the previous sheet where you change function `f` as follows. Note that our initial state now becomes `x0 = [0, 0, 0]`.

```

1 def f(x, t):
2     m = 1000
3     k = 50
4     Kp = 100
5     Ki = 5 # Note that the integral controller needs a third state
6     r = 20
7     F = Kp * (r - x[1]) + Ki * x[2]
8     der_x = x[1]
9     der_v = -k / m * x[1] + F / m
10    der_I = (r - x[1])
11    return [der_x, der_v, der_I]
```

Figure 2 shows a plot of the velocity v with gains of $K_p = 100, K_I = 5$. Observe that the PI controller no longer has a steady-state error (which was our intention with introducing the integral controller).

- (e) The hybrid component is visualized in Figure 3. Note that we reset the integral error each time the PI controller is re-activated. Moreover, the guards are triggered by the values 50 and 100. As invariants, instead, we used 45 and 105 to ensure that numerical algorithms like the Runge-Kutta method can be used to generate valid executions of the hybrid component. [For the mathematically inclined: Can you argue why?]
- (f) To implement this hybrid controller, first note that we cannot identify the mode with just the continuous states. For example, the state where $y(t) - x(t) = 75$ is a valid state for both Mode A as well as Mode B. Hence we introduce a global variable `mode` outside `f`¹. A first implementation of the hybrid controller could be as follows, where we implement the hybrid behavior in the function `f`. Denoting by F_h the force with which the car is being accelerated when the PI controller is inactive, the code below implements the desired hybrid cruise controller when $F_h = 0$.

```

1 mode = 1
2
```

¹We don't pass the value of `mode` as input and put of the `f` function so we can (hopefully) keep the implementation of the numerical solver unchanged.

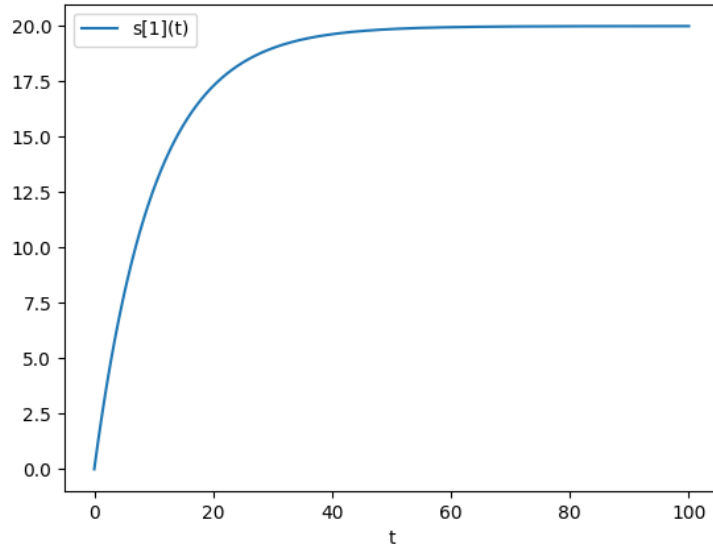


Figure 2: Car model state trajectory with PI controller $K_p = 100, K_I = 5$.

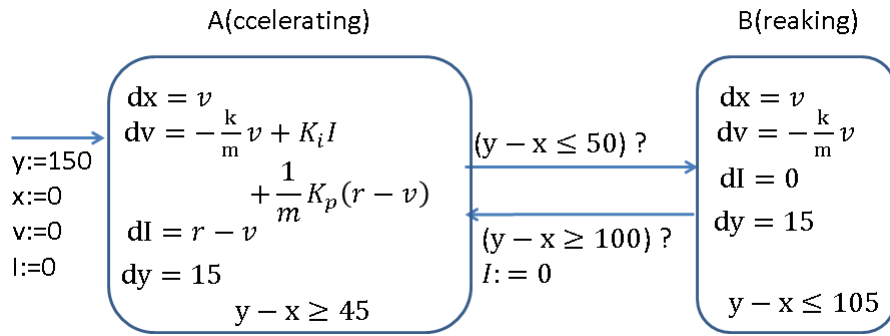


Figure 3: Hybrid cruise controller component.

```

3 def f(x, t):
4     global mode
5
6     m = 1000
7     k = 50
8     Kp = 100
9     Ki = 5 # Note that the integral controller needs a third state.
10    r = 20
11    Fh = 0
12
13    y_front = 150 + 15 * t
14
15    # Check whether we have to switch mode.
16    if mode == 1 and y_front - x[0] < 50:
17        mode = 2
18    if mode == 2 and y_front - x[0] > 100:

```

```

19     mode = 1
20
21     # Get the force depending on the mode.
22     if mode == 1:
23         F = Kp * (r - x[1]) + Ki * x[2]
24     else:
25         F = Fh
26
27     der_x = x[1]
28     der_v = -k / m * x[1] + F / m
29     der_I = (r - x[1])
30     return [der_x, der_v, der_I]

```

Figure 4 shows a plot of the velocity v with gains of $K_p = 100$, $K_I = 5$ (same as before) and $F_h = 0$. Observe that the controlled behavior is unstable! Note that for any other value of F_h the system remains unstable.

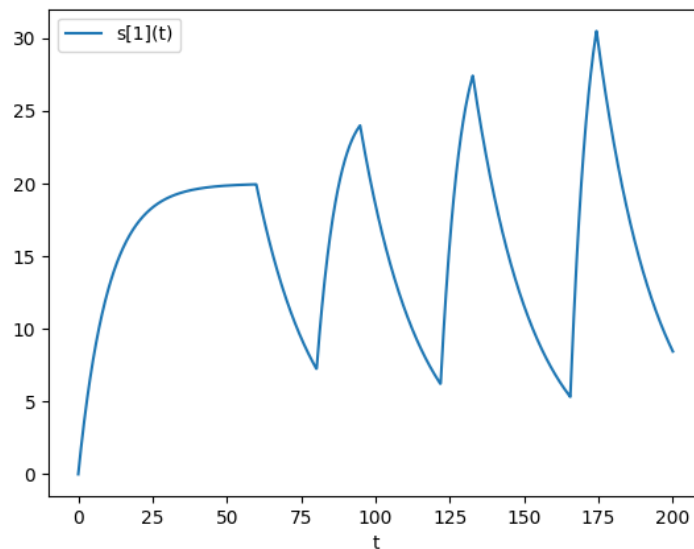


Figure 4: Hybrid car model state trajectory v with PI controller $K_p = 100$, $K_I = 5$ and $F_h = 0$.

The main cause of the instability is that the integral error $I = \int e \, dt$ keeps accumulating while the PI controller mode is not active. Ideally, we want to reset I every time we switch back to the PI mode. We can achieve this by separating the discrete mode switching from the continuous dynamics in `f`. Hence the numerical solver needs to be adjusted slightly. An implementation could be as follows. We use `switch=None` to keep the numerical solver backwards compatible for pure dynamical systems.

```

1 def euler_method(f, x0, t0, h, N, switch=None):
2     t = [t0]
3     result = [x0]
4     for i in range(N):
5         xi = result[-1]
6

```



```

7     # Check for mode switching.
8     if switch is not None:
9         xi = switching(xi, t[-1])
10
11     der = f(xi, t[-1])
12     t.append(t[-1] + h)
13     step = []
14     for j in range(len(der)):
15         step.append(xi[j] + h * der[j])
16     result.append(step)
17     return t, result
18
19 def f(x, t):
20     global mode
21     m = 1000
22     k = 50
23     Kp = 100
24     Ki = 5    # Note that the integral controller needs a third state.
25     Fh = 0
26     r = 20
27
28     # Get the force depending on the mode.
29     if mode == 1:
30         F = Kp * (r - x[1]) + Ki * x[2]
31     else:
32         F = Fh
33
34     der_x = x[1]
35     der_v = -k / m * x[1] + F / m
36     der_I = (r - x[1])
37     return [der_x, der_v, der_I]
38
39
40 def switching(x, t):
41     global mode
42     y_front = 150 + 15 * t
43
44     # Check whether we have to switch mode.
45     if mode == 1 and y_front - x[0] <= 50:
46         mode = 2
47     if mode == 2 and y_front - x[0] >= 100:
48         mode = 1
49         # We switch back to PI controller, so we reset the integral
50         # term.
51         x[2] = 0
52     return x

```

Figure 5 (left) shows a plot of the velocity v with gains of $K_p = 100$, $K_I = 5$ and $F_h = 0$ but now resetting the integral term each time the car switches to the PI mode. As can be seen, this now ensures that the hybrid system is stable. Don't forget that a good cruise controller does not collide in with the car in front of it. Therefore, Figure 5 (right) shows a plot of the position x of the car and y of the car in front of it. As can be seen, the blue line never actually touches the red line, hence the cars do not collide. One can also confirm this by plotting the distance between the front car and the car with the hybrid cruise controller $y(t) - x(t)$ as

shown in Figure 6.

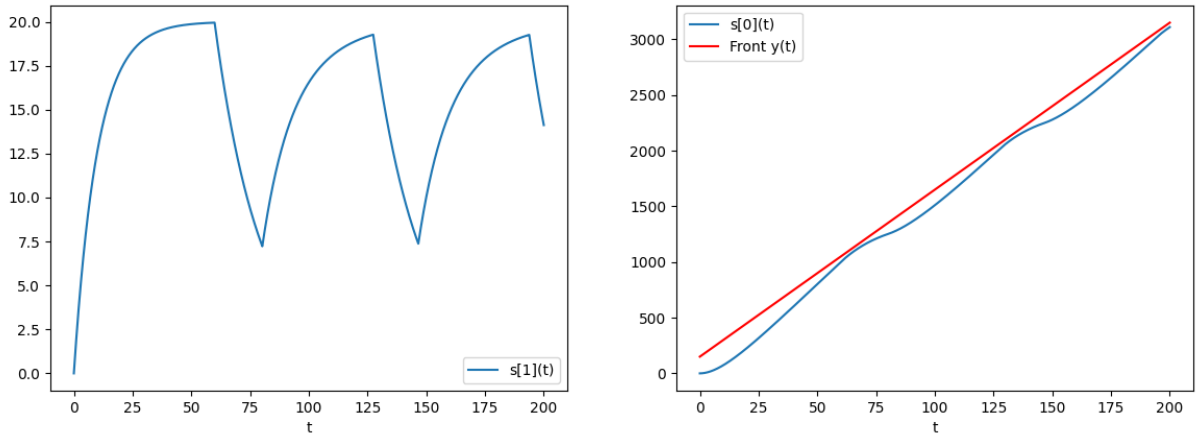


Figure 5: Hybrid car model state trajectory v (left) and the positions x and y (right), both with PI controller $K_p = 100$, $K_I = 5$ and $F_h = 0$ including the reset of the integral error term.

- (g) By considering different simulations for different values of F_h , it can be noted that we bump into the car ahead once F_h is around 650 or higher. Figure 7 shows the relative distance $y(t) - x(t)$.

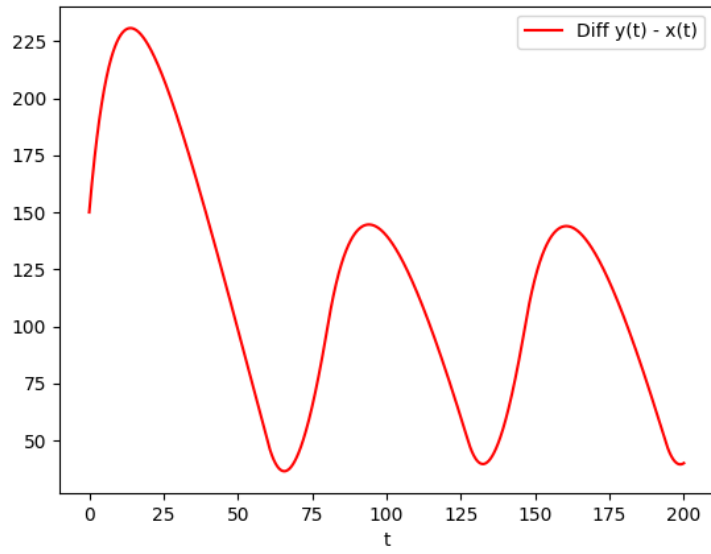


Figure 6: Distance between the car with the hybrid cruise controller and the car in front of it.

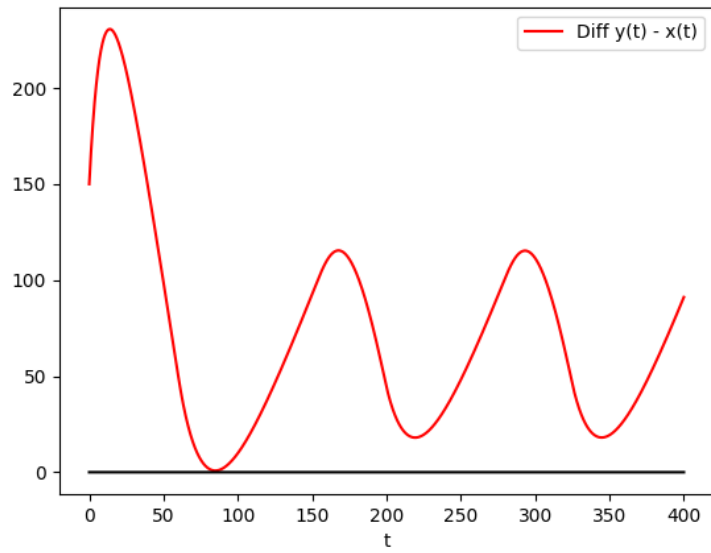


Figure 7: Relative distance $y(t) - x(t)$ between the cruise-controlled car and the car in front of it for a force $F_h = 650$ (N).